

Statistics

Python Maths

statistics

Python statistics Module

Python has a built-in module that you can use to calculate mathematical statistics of numeric data.

The statistics module was new in Python 3.4.

Import math

- # Import Library
`import statistics`

Mean()

- `# Import statistics Library`
`import statistics`

`# Calculate average values`
`print(statistics.mean([1, 3, 5, 7, 9, 11, 13]))`

- # Import statistics Library
`import statistics`

Calculate middle values
`print(statistics.median([1, 3, 5, 7, 9, 11, 13]))`

- # Import statistics Library
`import statistics`

Calculate the mode

`print(statistics.mode([1, 3, 3, 3, 5, 7, 7 9, 11]))`

- # Import statistics Library
`import statistics`

Calculate the standard deviation from a sample of data

`print(statistics.stdev([1, 3, 5, 7, 9, 11]))`

- `# Import statistics Library`
`import statistics`

`# Calculate the variance from a sample of data`
`print(statistics.variance([1, 3, 5, 7, 9, 11]))`

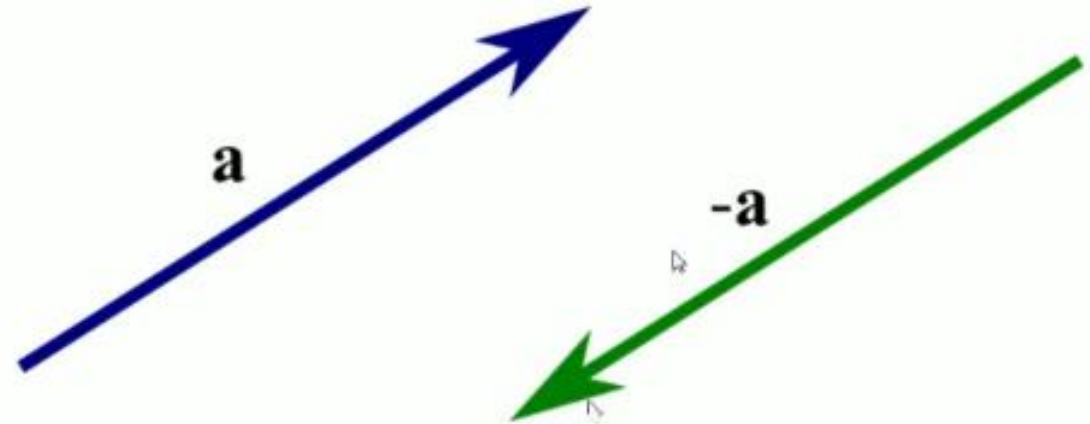
- A large variance indicates that the data is spread out, -
a small variance indicates that the data is clustered
closely around the mean.

Vectors

- Vectors are geometrical entities that have magnitude and direction. A vector can be represented by a line with an arrow pointing towards its direction and its length represents the magnitude of the vector.
- Therefore, vectors are represented by arrows, they have initial points and terminal points.
- The concept of vectors was evolved over a period of 200 years.
- Vectors are used to represent physical quantities such as displacement, velocity, acceleration, etc.

Vocabulary: Vector

A **vector** is a quantity that has both direction and *magnitude*, or size, and is represented in the coordinate plane by an arrow drawn from one point to another.



Video

- <https://www.youtube.com/watch?v=fqxWBj0cmDY&t=515s>

Vectors in Python

- Vectors are a fundamental concept in Python, especially useful for mathematical and scientific computations. You can work with vectors using libraries like NumPy, which provides efficient and easy-to-use functions for vector operations.

Create a vector

```
import numpy as np
```

```
# Create a vector  
vector = np.array([1, 2, 3])  
print("Vector:", vector)
```

Basic Operations

```
# Vector addition  
vector1 = np.array([1, 2, 3])  
vector2 = np.array([4, 5, 6])  
sum_vector = vector1 + vector2  
print("Sum:", sum_vector)
```

```
# Vector subtraction
```

```
diff_vector = vector1 - vector2
```

```
print("Difference:", diff_vector)
```

```
# Scalar multiplication
```

```
scalar_mult = vector1 * 2
```

```
print("Scalar Multiplication:", scalar_mult)
```

Magnitude and Normalization

Magnitude

```
magnitude = np.linalg.norm(vector)
```

```
print("Magnitude:", magnitude)
```

Normalization

```
normalized_vector = vector / magnitude
```

```
print("Normalized Vector:", normalized_vector)
```

Matrices

Matrices

A Matrix is an array of numbers:

$$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}$$

A Matrix

(This one has 2 Rows and 3 Columns)

Matrices

- Creating and manipulating matrices in NumPy is quite similar to working with vectors, but with additional functionalities tailored for multi-dimensional arrays.

Video

- https://www.youtube.com/watch?v=Wz1_Ma2cA-8&t=156s

Create a matrix

```
import numpy as np
```

```
# Create a matrix
```

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
print("Matrix:\n", matrix)
```

```
# Matrix addition
matrix1 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
matrix2 = np.array([[9, 8, 7], [6, 5, 4], [3, 2, 1]])
sum_matrix = matrix1 + matrix2
print("Sum:\n", sum_matrix)
```

```
# Matrix subtraction  
diff_matrix = matrix1 - matrix2  
print("Difference:\n", diff_matrix)
```

```
# Scalar multiplication  
scalar_mult = matrix1 * 2  
print("Scalar Multiplication:\n", scalar_mult)
```

```
# Matrix multiplication
product_matrix = np.dot(matrix1, matrix2)
print("Matrix Multiplication (np.dot):\n", product_matrix)

product_matrix_operator = matrix1 @ matrix2
print("Matrix Multiplication (@ operator):\n",
product_matrix_operator)
```

Transpose and Inverse

```
# Transpose
```

```
transpose_matrix = np.transpose(matrix1)
```

```
print("Transpose:\n", transpose_matrix)
```

```
# Inverse (only for square matrices)
```

```
inverse_matrix = np.linalg.inv(matrix1)
```

```
print("Inverse:\n", inverse_matrix)
```

Determinant and Eigenvalues

```
# Determinant
```

```
determinant = np.linalg.det(matrix1)
```

```
print("Determinant:", determinant)
```

```
# Eigenvalues and eigenvectors
```

```
eigenvalues, eigenvectors = np.linalg.eig(matrix1)
```

```
print("Eigenvalues:", eigenvalues)
```

```
print("Eigenvectors:\n", eigenvectors)
```

Resources

<https://www.w3schools.com/python/>

<https://www.cuemath.com/geometry/vectors/>

<https://www.mathsisfun.com/algebra/matrix-introduction.html>
